

Week 16: Bonus: categories and functors

Functional Programming in Lean

Learning goals

- ▶ State identity and associativity laws
- ▶ Connect a functor to a type and function mapping
- ▶ Prove an identity and composition law in Lean

Central idea

Composition gives a common account of functions. A functor preserves that structure.

Lean example

```
def compose (f :  $\alpha \rightarrow \beta$ )  
  (g :  $\beta \rightarrow \gamma$ ) :  $\alpha \rightarrow \gamma$  :=  
  fun x => g (f x)
```

```
theorem option_map_id (x : Option  $\alpha$ ) :  
  id <$> x = x := by  
  cases x <;> rfl
```

What to notice

- ▶ Objects can be types and arrows can be functions
- ▶ `Option.map` preserves identity and composition
- ▶ Law proofs add a contract to an implementation

Live coding

1. Prove associativity with `funext`
2. Prove `Option.map` identity by cases
3. State the composition law before revealing the proof

Pause and predict

What must `map` do to an identity function?

Exercise connection

Open `exercises/Week16.md` after the lecture.

The complete example is `Course/Week16.lean`; the fill-in version is `Templates/Week16.lean`.